

从 TD-MPC 到 TD-MPC2: 任务导向世界模型、规划与价值学习的统一理解

面向人工智能入门学习者的论文讲义

2026 年 7 月 4 日

一句话总览

TD-MPC 的核心思想是:

用任务导向 latent model 做短期规划 + 用 Q 函数补长期价值

TD-MPC2 的核心出发点是: TD-MPC 已经证明了“latent 世界模型 + MPC 规划 + TD value”这条路线很强, 但原始 TD-MPC 主要面向单任务, 容易依赖任务特定超参数, 并且直接放大模型规模时不一定更好。TD-MPC2 因此要解决三个问题:

1. 如何让算法在很多连续控制任务上稳定, 不需要每个任务重新调参;
2. 如何让同一个世界模型处理不同状态维度、动作维度和机器人形态;
3. 如何让模型规模和数据规模变大时, 性能也能稳定提升。

所以, TD-MPC2 不是推翻 TD-MPC, 而是把 TD-MPC 的思想做得更稳、更通用、更可扩展。

1 强化学习最小背景

连续控制任务通常可以写成一个马尔可夫决策过程:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \gamma).$$

其中:

符号	含义
$s \in \mathcal{S}$	状态，例如机器人关节角、速度、物体位置或图像观测
$a \in \mathcal{A}$	动作，例如每个关节的力矩或目标速度
$T(s, a)$	环境动力学，表示做动作后状态如何变化
$R(s, a)$	奖励函数，表示当前行为好不好
γ	折扣因子，控制未来奖励的重要程度

强化学习的目标是找到策略 π ，让长期回报最大：

$$\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right].$$

入门时可以把它理解成：机器人每一步都要选动作，但它不能只贪图眼前奖励，还要考虑这个动作会不会把自己带到一个未来更容易成功的位置。

2 Actor-Critic: 动作提出者与动作评分者

TD-MPC 和 TD-MPC2 都会用到 actor-critic 的基本思想。Actor 和 critic 的分工是：

- actor 或 policy 负责提出动作；
- critic 或 Q function 负责评价这个动作长期好不好。

确定性策略，例如 DDPG，可以写成：

$$a = \mu_{\theta}(s).$$

随机策略，例如 SAC，可以写成：

$$a \sim \pi_{\theta}(a|s).$$

Q 函数表示在状态 s 做动作 a 之后，未来长期回报的估计：

$$Q_{\phi}(s, a) \approx \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s, a_t = a \right].$$

一个很重要的理解是：

policy 负责提出动作，Q 负责给动作打长期分数。

SAC 还加入熵正则，鼓励策略不要太早变得确定：

$$\max_{\pi} \mathbb{E} [Q(s, a) + \alpha \mathcal{H}(\pi(\cdot|s))].$$

这里 $\mathcal{H}$ 是策略熵。熵越大，说明策略越有探索性。这会影响 TD-MPC2 的 policy prior 设计。

3 MPC、CEM 与 MPPI: 在脑内临时试动作

MPC, Model Predictive Control, 是一种“边走边规划”的控制思想。它不是一次性学出一个固定动作, 而是在当前状态下临时搜索未来几步动作序列:

$$\tau = (a_t, a_{t+1}, \dots, a_{t+H-1}).$$

如果有一个模型可以预测未来奖励, 那么可以给每条动作序列打分:

$$G(\tau) = \sum_{i=0}^{H-1} \gamma^i r_{t+i}.$$

然后选择分数最高的动作序列, 但只执行第一个动作 a_t 。下一步看到新状态后, 再重新规划。

为什么只执行第一个动作

因为模型预测未来一定有误差。越往后预测越不准。所以 MPC 每次只相信模型一小段时间, 执行一步后用真实环境反馈修正。这就像开车导航: 你可以看未来几公里路线, 但方向盘每秒都要根据真实路况重新调整。

CEM 与 MPPI 的区别

CEM 和 MPPI 都是采样式轨迹优化方法。

- CEM 更像只看 top- k 最好的轨迹, 再用这些精英轨迹更新动作分布;
- MPPI 更像让所有轨迹按好坏加权投票, 好轨迹权重大, 差轨迹权重小。

TD-MPC 系列就是把这种规划放到 learned latent model 里做。

4 TOLD: TD-MPC 的任务导向 latent 模型

TD-MPC 的世界模型叫 TOLD。它不试图预测完整未来世界, 例如像素图像。它只预测对控制任务有用的抽象状态、奖励、价值和动作建议。

TOLD 有五个模块:

$$\begin{aligned} z_t &= h_\theta(s_t), \\ z_{t+1} &= d_\theta(z_t, a_t), \\ \hat{r}_t &= R_\theta(z_t, a_t), \\ \hat{q}_t &= Q_\theta(z_t, a_t), \\ \hat{a}_t &= \pi_\theta(z_t). \end{aligned}$$

模块	作用
h_θ	把真实状态 s_t 编码成 latent state z_t
d_θ	在 latent 空间里预测下一步状态
R_θ	预测一步奖励
Q_θ	预测长期价值
π_θ	提供动作建议, 帮助规划器更快找到好动作

TOLD 的关键不是“完整复原世界”，而是“学一个对任务有用的脑内世界”。例如机械臂抓杯子时，模型不需要知道桌面纹理每个像素，但需要知道机械臂位置、杯子位置、动作后能不能更接近成功。

5 真实奖励 r_i 与预测奖励 $\hat{r}_i$

在强化学习交互中，真实奖励来自环境：

```
s_next, r_i, done, info = env.step(a_i)
```

也就是：

$$(s_i, a_i) \rightarrow (r_i, s_{i+1}).$$

这些数据会被存入 replay buffer：

$$(s_i, a_i, r_i, s_{i+1}).$$

因此：

- r_i 是真实环境返回的奖励，是训练 reward model 的标准答案；
- $\hat{r}_i = R_\theta(z_i, a_i)$ 是模型预测的奖励。

训练 reward model 的目标是：

$$R_\theta(z_i, a_i) \approx r_i.$$

6 TD-MPC 的训练与规划闭环

TD-MPC 的完整闭环如下：

1. 当前状态 s_t 进入系统；
2. encoder 得到 latent state $z_t = h_\theta(s_t)$ ；
3. CEM 或 MPPI 采样很多未来动作序列；

4. TOLD 在 latent 空间里模拟未来;
5. reward model 预测短期奖励;
6. Q function 预测终点之后的长期价值;
7. 选择预测回报最高的动作序列;
8. 只执行第一个动作;
9. 环境返回真实 r_t 和 s_{t+1} ;
10. 经验存入 replay buffer;
11. 用 replay buffer 继续训练 TOLD。

在 TD-MPC 中, 动作序列 τ 的评分不是只看短期奖励, 而是:

$$G(\tau) = \sum_{i=0}^{H-1} \gamma^i R_{\theta}(z_i, a_i) + \gamma^H Q_{\theta}(z_H, a_H).$$

这个公式非常重要。它说明:

- 前半部分由 reward model 负责, 回答“未来几步看起来怎么样”;
- 后半部分由 Q function 负责, 回答“几步之后的长期前途怎么样”。

因此 TD-MPC 可以用较短的规划 horizon, 但仍然不至于短视。

我的理解: 预训练与在线训练

可以精炼理解为:

- **单任务 TD-MPC2:** 更像“边交互边训练”。先用少量 seed transition 启动, 然后反复执行

真实交互 $\rightarrow$ 存入 replay buffer $\rightarrow$ 更新 $h, d, R, Q, p \rightarrow$ 重新规划.

- **多任务 TD-MPC2:** 可以理解为“先预训练, 再在线微调”。先用多任务 replay 数据训练通用 world model, 遇到新任务后再用真实交互继续更新。

一句话: TD-MPC2 不是用想象 rollout 训练自己, 真正进入 buffer 的仍是真实 transition; 想象 rollout 主要用于当前决策时的 MPC/MPPI 规划。

7 TD-MPC2 的论文出发点

TD-MPC 已经很强，但 TD-MPC2 认为它还有几个明显限制：

1. 主要成功集中在单任务学习；
2. 不同任务往往需要不同超参数；
3. 原始 TD-MPC 直接放大模型规模时，性能不一定稳定提升；
4. 多任务时，不同任务的状态维度、动作维度和机器人形态差异很大；
5. 奖励尺度差异会让训练不稳定；
6. latent 表示不受约束时，可能出现梯度爆炸。

所以 TD-MPC2 的出发点可以概括为：

让 TD-MPC 路线变得稳定、少调参、可多任务、可扩展。

TD-MPC2 在 104 个连续控制任务上测试，涵盖 DMControl、Meta-World、ManiSkill2 和 MyoSuite。它还训练了一个 317M 参数的多任务世界模型，在 80 个任务上展示了模型规模变大后性能继续提升。

8 OpenReview 视角下的创新点补充

OpenReview 的评审意见和作者回复让 TD-MPC2 的创新点更清楚。一个关键提醒是：TD-MPC2 的创新并不主要在于重新发明 MPC 规划器。评审人明确指出，TD-MPC2 的规划过程基本沿用 TD-MPC，真正的新意在于把 TD-MPC 改造成一个更稳、更少调参、更适合多任务和大模型扩展的系统。

创新一：从单任务 TD-MPC 走向多任务世界模型

OpenReview 中评审人首先强调的差异是多任务化修改。TD-MPC2 在所有模型组件中加入任务嵌入 e ：

$$z = h(s, e), \quad z' = d(z, a, e), \quad \hat{r} = R(z, a, e), \quad \hat{q} = Q(z, a, e), \quad a = p(z, e).$$

作者在回复中澄清：任务嵌入只是可学习向量，每个任务一个 embedding，和模型其他参数一起端到端训练。它没有额外的语义监督损失。任务语义之所以会在 embedding 空间中浮现，是因为模型必须依靠这些向量来区分不同任务、不同 embodiment 和不同动力学。

这点的创新意义是：TD-MPC2 不要求人为提供语言描述、任务标签语义或机器人结构知识，而是让模型通过控制数据自己学出任务关系。论文中任务 embedding 的 T-SNE 可视化也被评审人认为很有意思，因为语义相近或动力学相近的任务会自然靠近。

创新二：统一不同状态和动作空间

多任务连续控制的一个现实问题是，不同任务的状态维度和动作维度不一样。TD-MPC2 用 zero-padding 和 action mask 解决这个接口问题。

如果最大动作维度是 $A_{\max}$ ，某任务只有 A_k 维动作，则把动作补零到 $A_{\max}$ 维：

$$[a_1, \dots, a_{A_k}] \rightarrow [a_1, \dots, a_{A_k}, 0, \dots, 0].$$

同时使用 mask：

$$m = [1, \dots, 1, 0, \dots, 0].$$

训练 policy prior、计算 entropy、规划采样时只考虑有效动作维度。这看似是工程细节，但对多任务世界模型很关键。没有这个设计，一个统一模型就很难同时处理机械臂、四足机器人、肌肉骨骼手等不同动作空间。

创新三：离散回归让 reward/value 跨任务可比

OpenReview 中有评审人特别追问：为什么把奖励和价值预测做成离散回归？作者的回复让这个创新点更清楚。

问题在于，不同任务的 reward 和 value 尺度差异很大。如果直接用 MSE：

$$(\hat{r} - r)^2, \quad (\hat{q} - q)^2,$$

大尺度任务会主导梯度，小尺度任务会被淹没。

TD-MPC2 改用 101-bin 的离散回归和交叉熵。这不是为了让预测变粗糙，而是为了让不同任务的训练信号在数值尺度上更均衡。

作者还解释了替代方案为什么不理想：

- 把奖励二值化为 sparse success 会丢失很多 shaped reward 信号，尤其不适合 locomotion；
- 手动 reward normalization 需要知道每个任务奖励范围，大规模任务集里不可行；
- 自动运行统计归一化会引入非平稳性，并且容易受异常值影响；
- 即使单步 reward 被归一化，不同任务的 episode length、discount、稀疏性不同，value target 仍然可能尺度不同。

因此，离散回归的创新意义是：

它把 reward/value 尺度问题从任务级调参问题，变成统一的分类式预测问题。

创新四：稳定大模型训练的一组架构选择

OpenReview 里多位评审人关注 SimNorm、LayerNorm、Dropout、Mish 和 Q ensemble。作者回应说，TD-MPC2 虽然不是 Transformer，但确实借鉴了很多大模型训练中常见的稳定化设计：

- LayerNorm 让中间表示尺度更稳定；
- Mish 激活相比 ReLU/ELU 表现相近，但梯度更平滑；
- Dropout 用在 Q 函数中，帮助 Q 学习更稳；
- SimNorm 通过 softmax 单纯形约束 latent，使表示偏向稀疏；
- 多个 Q 函数形成 ensemble，降低过估计风险。

其中 SimNorm 的 OpenReview 解释尤其重要。作者把它类比为 VQ-VAE 离散码的软变体：VQ-VAE 用 one-hot 离散码表示 latent，SimNorm 则把 latent 划分为多个连续单纯形，每个子向量和为 1。直观地说：

VQ-VAE 是硬选择，SimNorm 是 softmax 松弛。

所以 SimNorm 不是简单归一化，而是在保留可微训练的同时，让 latent 表示带有稀疏和结构化倾向。

创新五：Q ensemble 采用随机子采样而不是全体取最小

TD-MPC2 默认学习 5 个 Q 函数。OpenReview 中有评审人问：为什么不直接对全部 5 个 Q 函数取最小值？

作者的回答是：对大集合取最小值可能过度保守，会把 target Q 的偏差传递给学习过程，损害数据效率；随机抽两个再取最小值，借鉴了 REDQ 一类方法中的思想，在减少过估计、保持数据效率和计算速度之间做折中。

因此这个设计的创新不是“越保守越好”，而是：

用适度保守的 Q target 稳定学习，同时避免全体取最小带来的过强低估。

创新六：最大熵 policy prior 让规划更容易扩展

TD-MPC2 的 policy prior 不是最终决策者，而是规划器的动作建议器。相比 TD-MPC 中更接近确定性策略加噪声的做法，TD-MPC2 使用最大熵目标训练 policy prior：

$$\max_p \mathbb{E} [\alpha Q(z, p(z)) + \beta \mathcal{H}(p(\cdot|z))].$$

OpenReview 中评审人也把这一点列为 TD-MPC2 相对 TD-MPC 的重要差异。它的意义是：规划器不是从完全随机的动作空间里盲搜，而是从一个保持探索性的动作先验附近搜索。这对高维动作空间尤其重要。

创新七：系统级可扩展性和开放资源

评审人和 meta-review 都认可：TD-MPC2 的贡献不仅是一个局部技巧，而是一个系统级改进。它在 104 个连续控制任务上评估，在 80 个任务上训练多任务模型，并展示能力随模型和数据规模增长。代码、数据、模型检查点以及训练 GPU 成本都公开，这也被 OpenReview 评审认为是加分项。

这里的创新可以理解为：TD-MPC2 证明了 TD-MPC 这条路线不只是一个单任务算法，而是有可能成为大规模连续控制世界模型的基础。

OpenReview 对创新边界的提醒

OpenReview 也提醒我们，不要把 TD-MPC2 的贡献讲得过满。更准确的说法是：

- 规划算法本身主要沿用 TD-MPC，不是 TD-MPC2 的主要新发明；
- 很多组件来自已有思想，例如 LayerNorm、Dropout、最大熵 RL、ensemble Q；
- TD-MPC2 的价值在于把这些设计仔细组合到 TD-MPC 中，并通过大规模实验验证它们互补有效；
- 方法仍主要面向连续动作空间，离散动作空间需要更换或重设规划方法，例如考虑 MCTS；
- 它继承 MPC 的部分缺点，例如决策时计算开销较高，以及在高度多模态转移中可能更困难；
- 多任务数据来自单任务 TD-MPC2 replay buffer，距离真实机器人领域更混杂的数据分布仍有距离。

所以从 OpenReview 的角度，TD-MPC2 的创新点最好这样表述：

TD-MPC2 的创新不是提出一个完全不同于 TD-MPC 的新范式，而是把 TD-MPC 系统化地改造成一个可稳定扩展到多任务、大模型和统一超参数设置的连续控制世界模型算法。

9 TD-MPC2 的世界模型目标

TD-MPC2 仍然学习一个 decoder-free 的隐式世界模型。所谓 decoder-free，是指它不预测原始观测，而是直接预测 latent、reward 和 value。

模型训练目标可以写成：

$$\mathcal{L}(\theta) = \mathbb{E} \sum_{t=0}^H \lambda^t \left[c_z \| \hat{z}_t - \text{sg}(h_\theta(s_t)) \|_2^2 + c_r \text{CE}(\hat{r}_t, r_t) + c_q \text{CE}(\hat{q}_t, q_t) \right].$$

三个部分分别对应：

损失项	含义
latent prediction	让模型在 latent 空间里预测未来状态
reward prediction	让模型预测真实环境奖励
value prediction	让模型预测 TD target 所代表的长期价值

其中 sg 表示 stop-gradient。它让目标 encoder 输出不反传梯度，避免模型自己追着自己动。

10 EMA target network 为什么稳定

TD target 是：

$$q_t = r_t + \gamma \bar{Q}(z_{t+1}, p(z_{t+1})).$$

这里 $\bar{Q}$ 是 EMA target network：

$$\bar{Q} \leftarrow \rho \bar{Q} + (1 - \rho) Q.$$

TD-MPC2 使用：

$$\rho = 0.99.$$

这表示每次更新时，target network 保留 99% 的旧参数，只吸收 1% 的当前 Q 参数。

为什么这有用？因为 TD 学习里目标本身依赖模型预测。如果直接用正在快速变化的 Q 来构造 target，模型就像在追一个不断移动、甚至自己制造出来的目标。

EMA 的效果是让目标变慢、变平滑：

- 降低单个 batch 噪声对 target 的影响；
- 减少 Q 估计误差在 bootstrapping 中快速放大；
- 让训练更接近一个目标相对稳定的监督学习问题。

代码形式如下：

```
@torch.no_grad()
def update_target(Q, Q_bar, rho=0.99):
    for p, p_bar in zip(Q.parameters(), Q_bar.parameters()):
        p_bar.data.mul_(rho)
        p_bar.data.add_((1 - rho) * p.data)
```

可以把 $\bar{Q}$ 理解成一个慢半拍的老师。学生 Q 每一步都在快速学习，老师 $\bar{Q}$ 会参考学生，但只慢慢修订标准答案。

我的理解：慢速锚点与误差累积

可以把 EMA target network 理解为给 TD 学习提供一个慢速锚点。因为 TD target 中当前要学习的价值依赖下一步的价值估计：

$$q_t = r_t + \gamma \bar{Q}(z_{t+1}, p(z_{t+1})).$$

也就是说， Q_t 的学习目标来自 Q_{t+1} 的预测。如果当前 Q 和下一步 Q 都同步快速变化，小的估计误差就可能沿着时间递推链条传播：

Q_{t+1} 估高 $\rightarrow Q_t$ 学到偏高 target $\rightarrow$ 后续更多 target 被带偏。

这就是 bootstrapping 中容易出现的误差累积和误差放大。EMA 的作用不是让 target 一步变得绝对准确，而是让用于构造 target 的 $\bar{Q}$ 慢慢跟随在线 Q ：

$$\bar{Q} \leftarrow 0.99\bar{Q} + 0.01Q.$$

因此，你可以把它理解为：

由于 TD 学习中的目标值由下一时刻的价值估计自举得到，当前 Q 与未来 Q 之间存在强耦合。如果所有 Q 估计都同步快速变化，小的估计误差会沿时间递推传播并被放大。因此 TD-MPC2 使用 EMA target network 作为慢速锚点，使 target 在局部时间窗口内相对稳定，让 Q 学习在小范围内平滑调整。它本质上是牺牲目标更新速度，换取训练稳定性和更低的误差累积风险。

所以，与其说 EMA 是简单地“以时间换精度”，不如说它是：

以目标更新速度换训练稳定性。

11 SimNorm: 约束 latent 表示

TD-MPC2 的一个关键稳定性设计是 SimNorm。假设 latent state z 是 512 维，SimNorm 维度 $V = 8$ 。模型把它分成：

$$512 = 64 \times 8.$$

每 8 维为一组，在组内做 softmax：

```
import torch
import torch.nn.functional as F

def simnorm(z, V=8, tau=1.0):
    shape = z.shape
    z = z.view(*shape[:-1], -1, V)
    z = F.softmax(z / tau, dim=-1)
    return z.view(*shape)
```

每一组 softmax 后都满足：

$$g_{i,j} \geq 0, \quad \sum_{j=1}^V g_{i,j} = 1.$$

这会带来两个效果：

1. latent 数值不能无限变大，降低梯度爆炸风险；
2. 每组倾向突出少数几个维度，形成更稀疏、更结构化的表示。

普通 latent 像一张自由草稿纸，可能越写越乱。SimNorm 像把 latent 填进很多个小概率分布盒子，每个盒子里总量固定，因此训练更稳定。

我的理解：有限要素槽位，而不是有限状态类别

可以把 SimNorm 直观理解为：一个环境中真正影响控制的抽象要素不是无限混乱的，模型可以把这些要素组织到有限个槽位中。在 TD-MPC2 的默认设置里，512 维 latent 被拆成 64 组，因此可以把它想成 64 个抽象因素槽位。每个槽位内部有 8 个候选模式，softmax 表示模型在这 8 个模式之间分配权重。

但这里需要特别注意：

不是说环境只有 64 种状态，也不是说只有 64 种要素。

更准确的说法是：

SimNorm 把 latent 表示拆成 64 个可组合的因素槽位，每个槽位做一次 soft 选择。

如果每个槽位接近 one-hot，整体组合能力大约类似于：

$$8^{64},$$

这是极其巨大的组合空间。所以 SimNorm 不是把环境粗暴压成 64 类，而是让模型用许多可组合的小结构来表示复杂状态。

因此，它主要不是通过减少参数量来减少训练量。latent 维度仍然是 512，模型计算量并不会因为 SimNorm 直接大幅下降。它真正减少的是表示空间的混乱程度：

- 限制 latent 数值无限变大；
- 让每组表示有固定总量；
- 让模型更倾向于使用稀疏、结构化的表示；
- 降低 dynamics、reward、Q 学习时的梯度不稳定风险。

所以这段理解可以整理成一句话：

SimNorm 可以理解为：我不让模型用完全自由的 512 维实数乱表示世界，而是让它把世界状态组织成 64 个抽象因素槽位；每个槽位在 8 个候选模式之间做 soft 选择。这不一定直接减少训练计算量，但会让表示更规整、更稳定，从而让训练更容易成功。

12 离散回归：解决奖励尺度差异

普通 reward 或 value 回归可能使用 MSE：

$$\mathcal{L}_{\text{MSE}} = (\hat{r} - r)^2.$$

但多任务中不同任务奖励尺度差异很大。一个任务奖励可能在 $[0, 1]$ ，另一个任务 return 可能接近 1000。直接 MSE 会让大尺度任务主导训练。

TD-MPC2 把 reward 和 value 预测变成离散回归。模型输出 101 个 bin 的 logits：

$$\hat{r} = \text{logits}_r \in \mathbb{R}^{101}, \quad \hat{q} = \text{logits}_q \in \mathbb{R}^{101}.$$

真实数值先经过变换并投影到 bin 上，然后用交叉熵学习：

$$\mathcal{L}_r = \text{CE}(\hat{r}, r), \quad \mathcal{L}_q = \text{CE}(\hat{q}, q).$$

直觉是：不强迫模型一开始就精确预测一个连续数字，而是先判断目标大概落在哪个区间。这对 value learning 特别重要，因为 value 本来就是带噪声的长期估计。

13 Q ensemble：减少过估计

Q-learning 容易有过估计问题。如果某个动作的 Q 值被偶然估得过高，规划器就会偏向它，导致错误被放大。

TD-MPC2 默认训练 5 个 Q 函数：

$$Q_1, Q_2, Q_3, Q_4, Q_5.$$

计算 target 时，随机抽两个 target Q 函数并取较小值：

$$q_t = r_t + \gamma \min(\bar{Q}_i(z_{t+1}, a'), \bar{Q}_j(z_{t+1}, a')),$$

$$a' = p(z_{t+1}).$$

这相当于两个老师打分时取更保守的那个，可以减少虚高价值对训练和规划的污染。

14 最大熵 policy prior

TD-MPC2 最终动作来自规划，但它仍然学习一个 policy prior:

$$a \sim p_{\theta}(\cdot|z, e).$$

policy prior 的任务不是替代规划，而是帮助规划。它告诉 MPPI：哪些动作比较可能是好动作。

TD-MPC2 使用最大熵思想训练 policy prior:

$$\max_p \mathbb{E} [\alpha Q(z, p(z)) + \beta \mathcal{H}(p(\cdot|z))].$$

第一项让策略倾向高价值动作；第二项让策略保持探索，不要过早塌缩到单一动作。

可以把 policy prior 理解成规划器的向导。它不是直接替规划器做最终决定，但能让规划器少在明显无用的动作区域浪费采样。

15 MPPI 规划：短期 reward 加长期 Q

TD-MPC2 使用 MPPI 做局部轨迹优化。每个决策步大致如下：

```
z_t = encoder(s_t)
initialize action distribution N(mu, sigma)

for iteration in range(6):
    sample 512 action sequences
    rollout each sequence in latent space
    score = predicted rewards + terminal Q
    update mu, sigma using elite / weighted trajectories

execute first action from the optimized sequence
```

规划目标可以写成：

$$\max_{a_t, \dots, a_{t+H}} \left[\sum_{i=0}^{H-1} \gamma^i R_{\theta}(z_{t+i}, a_{t+i}) + \gamma^H Q_{\theta}(z_{t+H}, a_{t+H}) \right].$$

TD-MPC2 默认 horizon 很短：

$$H = 3.$$

这看起来短，但并不短视，因为 terminal Q 负责估计三步之后的长期价值。也就是说：

MPC 管近处精细动作，Q 函数管远处长期趋势。

16 多任务：zero-padding 与 action mask

多任务学习会遇到一个工程难题：不同任务的状态维度和动作维度可能不同。
例如：

任务	状态维度	动作维度
小车	低维	2
机械臂	中维	7
仿生手	高维	39

TD-MPC2 的做法是把输入输出 pad 到最大维度。例如最大动作维度是 39，某任务只有 7 维动作：

$$[a_1, \dots, a_7] \rightarrow [a_1, \dots, a_7, 0, \dots, 0].$$

然后使用 action mask：

$$m = [1, \dots, 1, 0, \dots, 0].$$

训练和规划时只考虑有效动作维度。这可以避免模型在无效动作维度上制造虚假的 entropy 或预测误差。

此外，TD-MPC2 给每个任务一个可学习 task embedding：

$$e_k \in \mathbb{R}^{96}.$$

所有模块都可以条件化在这个任务向量上：

$$z = h(s, e), \quad z' = d(z, a, e), \quad \hat{r} = R(z, a, e), \quad \hat{q} = Q(z, a, e), \quad a = p(z, e).$$

这样同一个模型既共享通用控制知识，又能区分不同任务。

17 TD-MPC 与 TD-MPC2 的关系

方面	TD-MPC	TD-MPC2
核心思想	latent model + MPC + TD value	保留该主线并加强稳定性与扩展性
世界模型	TOLD, decoder-free	改进的隐式世界模型
latent 稳定性	latent 约束较弱	使用 SimNorm
reward/value 预测	连续回归为主	101-bin 离散回归
Q 函数	较少 Q 函数	默认 5 个 Q, 随机抽两个取 min
policy prior	确定性策略加噪声	最大熵随机策略
多任务支持	较有限	task embedding、zero-padding、action mask
超参数	更依赖任务调节	强调同一套超参数跨任务使用
扩展性	直接放大不一定稳定	展示 1M 到 317M 的 scaling

18 默认 5M TD-MPC2 的关键超参数

超参数	值	含义
Planning horizon	3	每次向前模拟 3 步
Population size	512	每轮采样 512 条动作序列
Elites	64	用最好的 64 条轨迹更新分布
Planning iterations	6	每个环境步迭代优化 6 次
Replay buffer	10^6	最多存一百万条经验
Batch size	256	每次训练采样 256 条数据
UTD	1	每收集一步数据更新一次模型
Latent dim	512	latent state 维度
Task embedding dim	96	多任务中任务向量维度
Q functions	5	Q ensemble 数量
Reward/value bins	101	离散回归 bin 数
Learning rate	3×10^{-4}	主网络学习率
Encoder LR	1×10^{-4}	encoder 学习率更小
Gradient clip	20	防止梯度过大

19 把整套系统想成机器人脑内结构

可以把 TD-MPC 和 TD-MPC2 想成一个机器人脑内系统：

部件	作用
环境	给真实状态转移和真实奖励
Replay buffer	存过去经验
Encoder	把真实状态压缩成 latent state
Dynamics model	在 latent 空间里想象未来
Reward model	判断短期行为好不好
Q function	判断长期前途好不好
Policy prior	给规划器动作建议
MPPI planner	在脑内试很多动作序列并选第一步执行

最重要的记忆方式是：

TD-MPC 不是直接学一个策略就结束，而是学一个能在 latent 空间里想象未来的模型，再每一步现场规划。

TD-MPC2 进一步把这个系统变成更稳定、更通用的版本：

TD-MPC2 = 稳定 latent 表示 + 稳定 value target + 保守 Q ensemble + 最大熵动作先验 + 多任务统一接口。

最后总结

如果只记住一条主线，应当是：

TD-MPC 解决的是如何用 learned latent model 做规划；TD-MPC2 解决的是如何让这条路线稳定、少调参、多任务、可扩展。

TD-MPC 的聪明之处在于，它不要求世界模型还原所有细节，只要求模型预测对控制有用的信息。TD-MPC2 的聪明之处在于，它发现真正阻碍这种方法规模化的不是单个公式，而是一组稳定性和工程鲁棒性问题：latent 会爆、reward 尺度不一、Q 会过估计、policy 会塌缩、多任务维度不齐。于是 TD-MPC2 用 SimNorm、离散回归、Q ensemble、EMA target、最大熵 policy prior、zero-padding 和 action mask，把这条方法链条补强。